



QUALITY ENGINEER PLAY BOOK

QA Engineers, Automation Testers, and Agile Teams



END-TO-END GUIDE FOR MASTERING QA PRACTICES, TOOLS, AND FRAMEWORKS IN REAL-WORLD
AGILE ENVIRONMENTS.

OCTOBER 4, 2025

Table of Contents

Section 1: Software Development Process & SDLC Models	2
Section 2: Software	3
Section 3: Agile Scrum Process	4
Section 4: Requirements Understanding	5
Section 5: Test Design	8
Section 6: Test Execution & Validation	10
Section 7: Java Programming Basics	11
Section 8: Java Fundamentals & Selenium Setup	14
Section 9: Selenium Fundamentals.....	16
Section 10: Selenium Setup with Maven & TestNG Integration.....	17
Section 11: DOM, Locators & Element Handling in Selenium	19
Section 12: Selenium Element Handling: Dropdowns, Iframes, Checkboxes, Radio Buttons, Alerts & Actions	21

Section 1: Software Development Process & SDLC Models

◆ SDLC Phases

The **Software Development Life Cycle (SDLC)** is a structured process that ensures software is delivered with quality, within budget, and on time. It consists of the following phases:

- **Requirements** – Gather business and functional needs from stakeholders.
- **Analysis** – Study requirements, identify feasibility, and refine scope.
- **Design** – Create system architecture, database schemas, flowcharts, HLDs/LLDs.
- **Development** – Write code using chosen programming languages and frameworks.
- **Testing** – Validate functionality against requirements through manual and automated testing.
- **Release** – Deploy the build to production environments.
- **Maintenance** – Provide support, bug fixes, and enhancements post-release.

◆ Waterfall Model

- A **linear, sequential model** where each phase must be completed before moving to the next.
- **Rigid:** No changes are allowed once requirements are locked.
- **Documentation-heavy:** Full BRDs/FSDs must be signed off before development begins.
- **Limitations:** Not flexible for evolving requirements, high risk of late defect discovery.

◆ Agile Manifesto

Agile addresses the limitations of Waterfall by promoting adaptability and collaboration. Its principles include:

- **Iterative delivery:** Working software is delivered in short cycles (sprints).
- **Face-to-face collaboration:** Direct communication between business and technical teams.
- **Motivated teams:** Empowered individuals with autonomy and accountability.
- **Responding to change:** Welcoming requirement changes, even late in development.

◆ Test Management Tools

To manage requirements, testcases, and execution efficiently, QA teams often use:

- **Rally**
- **Xray**
- **ReQTest**
- **TestRail**

These tools help track coverage, execution status, and defect metrics.

◆ User Story Example

In Agile, requirements are expressed as **user stories**:

“As a customer, I can do IMPS transaction.”

This format ensures clarity by defining the **user**, the **action**, and the **expected outcome**.

Section 2: Software

◆ Definition

- **Software** is a **collection of programs** designed to perform specific tasks or solve problems.
- A **program** is a set of logical instructions given to a computer to execute operations automatically.

◆ Categories of Software

- **Web-based Software**
 1. Runs in browsers (e.g., Facebook, Google, Salesforce, Banking apps).
 2. Requires internet access.
- **Desktop-based Software**
 1. Installed on personal computers (e.g., MS Office, Antivirus, Windows OS).
 2. Runs locally without internet dependency.
- **Mobile-based Software**
 1. Installed on smartphones via app stores (e.g., WhatsApp, Instagram).
 2. Optimized for mobile platforms.
- **Embedded Systems Software**
 1. Runs on specialized hardware (e.g., ATM card readers, routers, set-top boxes, medical devices).
 2. Typically, lightweight, and task-specific.

◆ Software Architecture Layers

- **Presentation Layer**
 1. The **UI (User Interface)** — what the user sees and interacts with.
 2. Example: Login page, buttons, forms.
 3. **UI Testing = Presentation Layer Testing.**
- **Application Layer**
 1. The **business logic** — processes user inputs, applies rules, and coordinates actions.
 2. Example: Validating login credentials, processing transactions.
- **Database Layer**
 1. The **data storage and retrieval** system.
 2. Example: Storing customer details, transaction history.

◆ Example: Contacts Manager

A simple **Contacts Manager** software demonstrates CRUD operations:

- **Create** → Add a new contact
- **Read** → Display contact information
- **Update** → Edit existing contact details
- **Delete** → Remove a contact

Section 3: Agile Scrum Process

◆ Scrum Overview

- **Scrum** is a lightweight development framework that combines the **Agile Manifesto**, the **Software Development Life Cycle (SDLC)**, and additional **guidelines**.
- It emphasizes iterative delivery, collaboration, and adaptability to changing requirements.

◆ Roles in Scrum

Each role has defined responsibilities to ensure smooth delivery:

- **PO – Product Owner**
Owns the product vision, manages the backlog, prioritizes features, and accepts/rejects sprint outcomes.
- **SM – Scrum Master**
Facilitates the Scrum process, removes blockers, and coaches the team on Agile practices.
- **Dev – Developers**
Build features, write code, perform unit/component testing, and integrate modules.
- **Testers (QA Engineers)**
Validate functionality, design and execute test cases, and report defects.
- **DevOps – Development & Operations Engineers**
Manage CI/CD pipelines, deployments, and environment stability.
- **TM – Test Manager**
Oversees QA strategy, metrics, and reporting across sprints.
- **PM – Project Manager**
Aligns project timelines, budgets, and stakeholder expectations.
- **SMEs – Subject Matter Experts**
Provide domain knowledge and clarify requirements.

◆ Scrum Ceremonies (Meetings)

- **Sprint Planning Meeting** – Define sprint goals, select backlog items, and estimate effort.
- **Daily Standups** – 15-minute sync on progress, blockers, and next steps.
- **Sprint Review** – Demonstrate completed work to stakeholders for feedback.
- **Sprint Retrospective** – Reflect on what went well, what can improve, and agree on action items.

◆ Environments in Scrum Delivery

Software passes through multiple environments before production:

- **Dev – Development Environment**
Developers integrate and test code modules.
 - **QA – Quality Assurance Environment**
Testers validate builds against test cases.
 - **UAT – User Acceptance Testing Environment**
Business users validate functionality against requirements.
 - **Staging Environment**
Pre-production environment for final checks and near-real simulations.
 - **Production Environment**
Live environment accessible to end-users.
-

◆ Requirement Documentation

Different artifacts capture requirements at varying levels:

- **BRD – Business Requirements Document**
Captures high-level business needs and objectives.
- **FSD – Functional Specification Document**
Translates business requirements into detailed functional flows, wireframes, and system behaviour.
- **User Stories**
Simplified, testable requirements written in the format:
“As a [role], I want [feature], so that [benefit].”

◆ Example: Admin Module

The **Admin Module** can be broken down into user stories and test scenarios:

- **Create a Branch**
- **Update a Branch**
- **Delete a Branch**

Each story is linked to acceptance criteria and corresponding test cases.

◆ QA Responsibilities in Scrum

Quality Assurance ensures delivery meets expectations by:

- **Requirement Understanding** – Participate in backlog grooming, clarify doubts with PO/SMEs.
- **Test Design** – Prepare test scenarios, test cases, and review them with peers.
- **Validation** – Execute test cases on builds, log results, and verify fixes.
- **Defect Management** – Report, track, and retest defects until closure.
- **Reporting** – Share test coverage, defect metrics, and sign-off readiness with stakeholders.

Section 4: Requirements Understanding

◆ Requirements Understanding

- **Requirement Sources:**
 - **BRD – Business Requirements Document**
 - **FSD – Functional Specification Document**
 - **User Stories** (Agile format)
- **Clarification:** Requirements are validated with **onsite teams or clients** to resolve ambiguities.

◆ Test Design

- **Test Scenario Preparation** – High-level conditions derived from requirements.
- **Test Case Preparation** – Step-by-step validation with inputs and expected results.
- **Test Design Review** – Peer review to ensure coverage, accuracy, and traceability.

◆ Test Scenario Definition

- A **test scenario** is a **requirement expressed in testable form**.
- **Hierarchy:**
- Epic → Features → User Stories → Test Scenarios → Test Cases
- **User Stories** break functionalities into smaller, testable parts.
- **Work Allocation Example:**
 - 5 user stories → 3 testers
 - Tester 1 → 2 user stories

- Tester 2 → 2 user stories
- Tester 3 → 1 user story

◆ **Module-Wise Breakdown**

1. Admin Module (Epic)

Features: Branches, Users, Roles, Employees

- **Branches (User Stories):**
 - Create a branch
 - Update a branch
 - Delete a branch

2. Customer Module

- **User Stories:**
 - Login
 - Register
 - Forgot password
 - Add payee
 - IMPS transaction
- **Traceability ID Format:**
TSNUM_USNO_MODULE_SPRINT_ProjectName

3. Statements Module

- **User Stories:**
 - PDF Download
 - Excel Download
 - Email Statements
 - View Statements

4. WhatsApp Chat Module

- **User Stories:**
 - Text message
 - Voice message
 - Video message
 - Voice call
 - Video call
 - Group voice call
 - Group video call

5. Travellers Module

- **Features:** Agent → Customer → Journey Booking
 - **User Stories:**
 - Agent login
 - Create customer form (fields: name, age, gender, location, mobile, email, date, return, etc.)
 - Book journey
 - Generate quotation (insurance details, amount, terms & conditions)
 - Download quotation (PDF)
 - Email quotation
-

6. Product Cancel Module

- **User Stories:**
 1. Date filter
 2. Product display with cancel button
 3. Cancel confirmation box
 4. Confirmation message
-

7. Order Product Module

- **User Stories:**
 - Select product
 - Add to cart
 - Buy now
 - Enter shipping address
 - Select payment mode (Net banking, Cards, UPI)
 - Confirm order
 - Payment confirmation
-

8. RedBus Booking Module

- **User Stories:**
 1. From location selection
 2. To location selection
 3. Departure date selection
 4. Return date selection
 5. Confirm travel
 6. Login / Register
 7. Payment method selection
 8. Net banking
 9. Cards
 10. UPI
 11. Payment confirmation
 12. Booking confirmation via SMS/Email
-

◆ Conceptual Note

- **Software = Collection of Pages**
- **Page Composition:** Pages may embed other pages or content via **iframes**.

Section 5: Test Design

◆ Test Design Overview

- **Test Scenarios** → High-level conditions derived from requirements.
- **Testcases** → Testable pieces within a scenario, with detailed steps, data, and expected results.
- **Hierarchy:**
 - Epic → Feature → User Story → Test Scenario → Testcase
- A **testcase** validates a **scenario**, which in turn validates a **user story**, feature, and epic — ensuring full requirements coverage.

◆ Example Test Scenarios & Testcases

Scenario 1: About Us Page

- **Requirement:** User clicks on About Us page → System should display About Us information.
- **Testcase:** Verify that the system displays About Us information when user clicks on the About Us link.

Scenario 2: Admin Login

- **Requirement:** User logs in with admin credentials → System should display admin dashboard.
- **Testcase:** Verify that the system displays the admin dashboard page when admin logs in with valid credentials.

Scenario 3: Country Dropdown

- **Requirement:** User clicks on Country field → System should display all countries.
- **Testcase:** Verify that the system displays all countries when admin clicks on the country dropdown field.

Scenario 4: Create Branch

- **Requirement:** User clicks on Submit in Create Branch page → System should create new branch in database.
- **Testcase:** Verify that the system creates a new branch in the database when admin clicks on Submit.

◆ Testcase Components

- **Testcase ID:** Unique identifier (format: TCNO_TSNO_USNO_MODULE_SPRINT_PROJECT)
- **Title:** Short description of the testcase
- **Pre-requisites:** Conditions required before execution
- **Test Steps:** Step-by-step actions to validate functionality
- **Test Data:** Input values required for execution
- **Expected Output:** Anticipated system behaviour
- **Actual Output:** Observed behaviour during execution
- **Status:** Pass/Fail based on comparison of expected vs actual
- **Comments:** Notes on deviations, issues, or observations

◆ Detailed Example: Create Branch Testcase

- **ID:** TC_001_CreateBranch
Title: Verify branch creation with valid inputs
- **Pre-requisites:**
 - Browser installed
 - Server up and running
 - Valid admin credentials available

- **Test Steps & Data:**
 - Launch browser → *Chrome*
 - Navigate to login page → <https://login.abc.com>
 - Login with admin credentials → *admin / admin123*
 - Click on Branches menu → *NA*
 - Click on New Branch menu → *NA*
 - Enter branch details → *branchName, address, city, country*
 - Click on Submit → *NA*
 - **Validation Rules:**
 - Branch name: alphanumeric, 6–40 characters
- **Expected Output:**
 - Browser launches
 - Login page displays with fields
 - Admin dashboard displays
 - Branches page displays
 - Create Branch form displays
 - System accepts form data
 - System validates and saves branch in database
- **Actual Output:**
 - Browser launched
 - Login page displayed
 - Admin dashboard displayed
 - Branches page displayed
 - Create Branch form displayed
 - Form accepted data
 - Data validated and saved in database
- **Status:** Passed
- **Comments:** Login failed once despite correct credentials (intermittent issue).

◆ Estimation & Story Points

- **Formula:** $1 * 10 * 8 = 80H$ (example calculation)
- **Breakdown:**
 - Test scenario design → 10h
 - Testcase design → 15h
 - Execution → 15h
 - Retesting → 5h
 - Artifacts (logs, reports) → 5h
 - **Total = 50h**
- **Story Point Mapping:**
 - 1 SP = 8h
 - 40h = 5 SP
 - 80h = 10 SP

◆ Roles & Tools

- **Lead:** Uses Rally, Azure DevOps, Jira for tracking.
 - **QA Analyst:** Responsible for test design, execution, retesting, and maintaining artifacts.
-

Section 6: Test Execution & Validation

◆ Overview

Test validation ensures that developed functionality meets the expected behaviour defined in testcases. Execution involves running testcases on builds, logging results, reporting defects, and retesting fixes.

◆ Execution Workflow

1. Execute Testcases on Build

- **Build Delivery:** Code is compiled and delivered by the development team.
- **Validation:** QA executes testcases against the build to verify functionality.
- **Outcome:**
 - If functionality works → log in **Test Log Document**
 - If functionality fails → raise **Defect Report Document**

2. Defect Report Document

- **Purpose:** Proof that a specific functionality is not working as expected.
- **Audience:** Developers
- **Contents:**
 - Defect ID
 - Summary
 - Steps to reproduce
 - Screenshots or video evidence
 - Environment details
 - Severity and priority
- **Tool Examples:** [Lightshot for screenshots](#)
- **Volume:** 100 defects → 100 individual defect reports

3. Test Log Document

- **Purpose:** Proof that functionality is working as expected.
- **Audience:** Product Owner, Test Manager, Stakeholders
- **Contents:**
 - Testcase ID
 - Title
 - Steps executed
 - Test data
 - Expected vs Actual output
 - Status
- **Volume:**
 - 300 testcases → ideally consolidated into **one document**
 - Avoid 300 separate files unless required by audit

◆ Defect/Bug Life Cycle

- Find Defect → Report to Developer → Fix → Retest → Confirm Fix → Close
- **Severity Levels**
 - **Critical:** Showstoppers, blockers, system crashes
 - **High:** Major functionality broken
 - **Medium:** Partial functionality issues
 - **Low:** Cosmetic or minor glitches

- **Priority Handling**
 - **Immediate Fix:** Critical blockers
 - **Postpone/Defer:** Valid but non-urgent defects
 - **Reject:** Invalid or non-reproducible issues
- **Tools Used**
 - **Azure DevOps**
 - **Jira**
 - **Rally (CA Agile Central)**

◆ **Sprint & Release Planning**

- **Release Cycles**
 - **2 Months, 3 Months, 6 Months**
 - Example: 3-Month Release → 9 Sprints
- **Sprint Rules**
 - **In-Sprint Changes:** Not allowed after Sprint Planning
 - **Mid-Release Changes:** Allowed in later sprints with PO approval
 - **Sprint Outcome:** Code moves to **Production** after final sprint
- **Example Timeline**
 - **Sprint 5:** Change request raised
 - **Sprint 9:** Change accommodated
 - **Production Deployment:** Post Sprint 9

Section 7: Java Programming Basics

◆ **Overview**

- **Java** is a **platform-independent, object-oriented programming language**.
- Created by **James Gosling** and the **Green Team** at Sun Microsystems (later acquired by Oracle).
- Principle: “**Write Once, Run Anywhere**” — compiled bytecode runs on any system with a Java Virtual Machine (JVM).

◆ **JDK – Java Development Kit**

The **JDK** is the complete toolkit for Java development. It includes:

- **Java Libraries** – Pre-built classes and APIs.
- **Compiler (javac)** – Converts source code into bytecode.
- **JVM (Java Virtual Machine)** – Executes bytecode on any platform.

◆ **Data Types in Java**

Java supports **primitive data types**:

- **byte** – 8-bit integer
- **short** – 16-bit integer
- **int** – 32-bit integer
- **long** – 64-bit integer
- **float** – 32-bit floating-point
- **double** – 64-bit floating-point
- **char** – 16-bit Unicode character
- **Boolean** – true/false values

◆ Variables

- **Definition:** Variables are named memory locations used to store data.
- **Declaration & Initialization Examples:**
 - `int a = 10; // integer variable`
 - `float f = 12.34f; // floating-point variable`
 - `String name = "Clifford"; // reference type variable`
- **Types of Variables:**
 - **Local Variables** – Declared inside methods, accessible only within them.
 - **Instance Variables** – Declared inside a class but outside methods, tied to objects.
 - **Static Variables** – Declared with static, shared across all objects of a class.

◆ Identifiers

- **Definition:** Names given to variables, methods, classes, interfaces, and packages.
- **Rules:**
 - Must begin with a letter, underscore `_`, or dollar `$`.
 - Cannot start with a digit.
 - Cannot use reserved **keywords** (e.g., `class`, `public`).
 - Case-sensitive (`totalAmount` ≠ `TotalAmount`).
 - No spaces or special characters (except `_` and `$`).
- **Examples:**
 -  Valid: `customerId`, `_result`, `$cache`, `sum2`
 -  Invalid: `2sum`, `first-name`, `total amount`, `class`
- **Naming Conventions:**
 - Variables & methods → camelCase (e.g., `orderTotal`, `calculateTax()`)
 - Classes & interfaces → PascalCase (e.g., `PaymentService`)
 - Constants → UPPER_SNAKE_CASE (e.g., `MAX_RETRY_COUNT`)
 - Packages → lowercase, dot-separated (e.g., `com.bankapp.admin`)

◆ Keywords

- **Definition:** Keywords are reserved words in Java with predefined meanings
- Java has **70+ reserved keywords** (e.g., `class`, `public`, `static`, `void`, `if`, `else`, `switch`, `try`, `catch`).
- These cannot be used as identifiers.

◆ Step 1: Create a Project

- **In Eclipse:**
File → New → Project → Java Project → Name → Finish
- **Build process:** Source file → Compile → Bytecode → JVM → Output
- **Folders:**
 - `src` → Source programs
 - `bin` → Compiled `.class` files

◆ Step 2: Create a Package

- In Eclipse:
Project → `src` → New → Package → Package Name → Finish

◆ Step 3: Create a Class

- In Eclipse:
Project → Package → New → Class → Class Name → Finish
- **Class = Container for functionality**
- Functionality must be written inside **methods**.

Notes:

- To print **static data** → use double quotes ("Hello").
- To print **dynamic data** → no quotes (e.g., variable names).
- Static + Dynamic → Appended automatically.

◆ **Methods in Java**

- **Definition:** A method is a block of code that performs a specific task.
- **Syntax:**
- AccessModifier static returnType methodName(arg1, arg2...) {
- // code
- }
- **AccessModifier:** public, private, protected, or default (mandatory).
 - **static:** Optional keyword.
 - **returnType:** Type of output produced (mandatory).
 - **methodName:** Identifier for the method (mandatory).
 - **arguments:** Inputs to the method (optional).

◆ **Method Access**

- **By Object (instance methods):**
 - MyClass obj = new MyClass();
 - obj.myMethod();
- Objects can access **both static and non-static methods**.
- **By Class (static methods only):**
 - MyClass.myStaticMethod();
- Classes can access **only static methods**.

◆ **Frameworks Commonly Used in QA**

- **TestNG** – Testing framework for unit, integration, and automation tests.
- **JUnit** – Lightweight unit testing framework.
- **BDD (Behavior-Driven Development)** – Frameworks like Cucumber for natural-language test writing.
- **Hybrid Frameworks** – Combine multiple approaches (e.g., Data-Driven + Keyword-Driven + POM).

◆ **Tools in the Java Ecosystem**

- **Git** – Version control system.
- **Maven** – Build automation and dependency management.
- **Jenkins** – Continuous Integration/Continuous Deployment (CI/CD).
- **Selenium Grid** – Distributed test execution across browsers and machines.

◆ **POM – Page Object Model**

- **Page Object Model (POM)** is a **design pattern** in test automation.
- Each page of the application is represented by a separate class.
- **Benefits:**
 - Improves **code reusability**
 - Enhances **readability and maintainability**
 - Reduces **duplication of locators and methods**

Section 8: Java Fundamentals & Selenium Setup

◆ 1. Method Arguments

- **Definition:** Inputs passed to a method to perform operations.
- **Syntax Example:**

```
public void greetUser(String name) {
    System.out.println("Hello, " + name);
}
```
- **Usage:**
 - Inside main() method
 - From other classes via object or static call

◆ 2. Return Type

- **Definition:** The type of output a method produces and shares with other methods.
- **Examples:**

```
public int add(int a, int b) {
    return a + b;
}

public String getStatus() {
    return "Active";
}
```
- **Common Return Types:** int, String, Boolean, void (no return)

◆ 3. Conditions in Java

✅ if-else Condition

- **Syntax:**

```
if (condition) {
    // code if true
} else {
    // code if false
}
```
- **Example:**

```
if (powerAvailable) {
    System.out.println("Class will be conducted.");
} else {
    System.out.println("Class is cancelled.");
}
```

✅ else-if Ladder

- **Syntax:**

```
if (condition1) {
    // code
} else if (condition2) {
    // code
} else {
    // fallback code
}
```

✅ switch Statement

- **Syntax:**

```
switch (day) {
```

```

case "Monday":
    // code
    break;
case "Tuesday":
    // code
    break;
default:
    // fallback
}

```

✓ String Comparison

- equals() → Case-sensitive comparison
- equalsIgnoreCase() → Case-insensitive comparison
- "Chrome".equals("chrome"); // false
- "Chrome".equalsIgnoreCase("chrome"); // true

◆ 4. Selenium Setup & Examples

✓ Setup Steps

1. **Install Java JDK**
2. **Install Eclipse IDE**
3. **Add Selenium WebDriver JARs to project**
4. **Create Java Project → Package → Class**
5. **Write Selenium code inside methods**

✓ Example: Launch Chrome & Open URL

```

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class LaunchBrowser {
    public static void main(String[] args) {
        System.setProperty("webdriver.chrome.driver", "path/to/chromedriver.exe");
        WebDriver driver = new ChromeDriver();
        driver.get("https://www.google.com");
    }
}

```

✓ Example: Login Validation

```

public boolean validateLogin(String username, String password) {
    if (username.equals("admin") && password.equals("admin123")) {
        return true;
    } else {
        return false;
    }
}

```


Section 9: Selenium Fundamentals

◆ What Is Selenium?

- **Selenium** is a powerful **Java-based automation library** used to simulate manual actions on web applications.
- It provides methods to interact with **web elements** like buttons, checkboxes, dropdowns, and tables.
- **Automation = Manual Action → Code**

◆ Software → Pages → Elements → Actions

Layer	Description
Software	A collection of pages (e.g., Login, Dashboard, Profile)
Page	A collection of elements (e.g., buttons, fields, links)
Elements	UI components like radio buttons, checkboxes, dropdowns, tables, etc.
Actions	Operations performed on elements: Click, Enter, Select, Drag & Drop, etc.

◆ Selenium Architecture

Selenium Code → Browser Driver → Browser

- **Selenium cannot talk to browsers directly.**
- It uses **browser drivers** as intermediaries:
 - ChromeDriver → Chrome
 - GeckoDriver → Firefox
 - OperaDriver → Opera
- Drivers are downloaded from selenium.dev/downloads

◆ Selenium as a Java Library

- Selenium is a **set of Java classes** containing reusable **methods**.
- Learning Selenium = Learning how to use these methods effectively.
- Selenium code is written inside a **Java Project**, then packaged as a .jar file for reuse.

◆ Common Automation Steps in Testcases

- **Launch Browser**
 - `WebDriver driver = new ChromeDriver();`
- **Navigate to AUT (Application Under Test)**
 - `driver.get("https://example.com");`
- **Interact with Elements**
 - `driver.findElement(By.id("username")).sendKeys("admin");`
 - `driver.findElement(By.id("submit")).click();`
- **Close Browser**
 - `driver.quit();`

◆ **Static vs Non-Static in Java (Selenium Context)**

Concept	Description
Static	Memory allocated at class load time. Shared across all instances.
Non-Static	Memory allocated only when an object is created. Used for instance methods.
Class Usage	A class may contain 100+ methods, but only a few are used per test.
Object Usage	Creating an object allocates memory for non-static methods.

Section 10: Selenium Setup with Maven & TestNG Integration

◆ **1. Selenium Setup Overview**

- **Selenium** is a Java-based automation library used to simulate manual actions on web applications.
- It provides methods to interact with UI elements like buttons, dropdowns, tables, and forms.
- **Automation = Manual Action → Code**

◆ **2. Browser Drivers**

- Selenium communicates with browsers via **browser drivers**:

Browser	Driver	Download Source
Chrome	ChromeDriver	selenium.dev/downloads
Firefox	GeckoDriver	selenium.dev/downloads
Opera	OperaDriver	selenium.dev/downloads

- **Architecture:**
 - Selenium Code → Browser Driver → Browser

◆ **3. Java Project Setup (Manual)**

- **Create Java Project** in Eclipse
- **Create folders** for:
 - Business logic (src/main/java)
 - Assets (src/main/resources)
 - Unit testing (src/test/java)
 - Test assets (src/test/resources)
- **Download and attach JARs:**
 - Selenium WebDriver → selenium.dev
 - TestNG → testng.org
 - Apache POI → apache.org/poi
 - Extent Reports → extentreports.com
- **Execute testcases** using Eclipse or command line

◆ **4. Maven Project Setup (Recommended)**

- **Maven** is a build management tool that automates dependency handling and folder creation.
- ✓ **Benefits**
 - Auto-creates standard folder structure
 - Downloads required JARs via pom.xml
 - Supports CI/CD integration (e.g., Jenkins)
 - Enables command-line execution

✓ **Maven Folder Structure**

Folder	Purpose
src/main/java	Business logic (Selenium code)
src/main/resources	Assets (drivers, APKs)
src/test/java	Unit testing code (TestNG testcases)
src/test/resources	Test assets (XML, Excel, JSON, etc.)

✓ **POM (Project Object Model)**

- **Group ID:** Package name (e.g., com.hdfc)
- **Artifact ID:** Project name (e.g., hdfc-automation)
- **Dependencies:**

```
<dependencies>
  <dependency>
    <groupId>org.seleniumhq.selenium</groupId>
    <artifactId>selenium-java</artifactId>
    <version>4.12.0</version>
  </dependency>
  <dependency>
    <groupId>org.testng</groupId>
    <artifactId>testng</artifactId>
    <version>7.9.0</version>
  </dependency>
  <!-- Add Apache POI, Extent Reports as needed -->
</dependencies>
```

◆ **5. TestNG Framework**

✓ **What is TestNG?**

- A **next-generation unit testing framework** for Java.
- Acts as the **skeleton** for test execution; Selenium components are the **organs**.

✓ **Key Features**

- **Annotations**
- **Properties**
- **Listeners**
- **Data Providers**
- **Assertions**

✓ **Common Annotations**

Annotation	Purpose
@BeforeSuite	Executes once before all tests
@AfterSuite	Executes once after all tests
@BeforeTest	Executes before <test> tag in XML
@AfterTest	Executes after <test> tag in XML
@BeforeClass	Executes before first method in class
@AfterClass	Executes after last method in class
@BeforeMethod	Executes before every @Test method
@AfterMethod	Executes after every @Test method
@Test	Marks a method as a testcase

✅ **Example: Setup & Teardown**

```
@BeforeMethod
public void setup() {
    // Launch browser, navigate to AUT (Application under test)
}

@AfterMethod
public void teardown() {
    // Close browser
}
```

Section 11: DOM, Locators & Element Handling in Selenium

◆ 1. DOM – Document Object Model

- **Definition:** The DOM is a **virtual representation of a web page**, structured as a tree of elements.
- **Analogy:**
 - **Page = Village**
 - **Elements = Houses**
 - **Address = Locator (ID, Name, Class, etc.)**
 - **Letter = Action (Click, Enter, Select)**
- Selenium uses the DOM to **find and interact with elements** by their unique addresses.

◆ 2. Locators in Selenium

- **Locators are methods from the By class used to identify elements on a page.**

Locator Method	Description	Example Usage
By.id()	Locates element by its id attribute	By.id("username")
By.name()	Locates element by its name attribute	By.name("username")
By.className()	Locates element by its class attribute	By.className("input")
By.tagName()	Locates element by HTML tag	By.tagName("input")
By.linkText()	Locates link by exact text	By.linkText("Forgot Password")
By.partialLinkText()	Locates link by partial text	By.partialLinkText("Forgot")
By.cssSelector()	Locates using CSS syntax	By.cssSelector("input#username")
By.xpath()	Locates using XPath expressions	By.xpath("//input[@id='username']")

🔗 [Selenium By Class Documentation](#)

◆ 3. Element Source Code Example

```
<input class="input r4 wide mb16 mt8 username"
      type="email"
      value=""
      name="username"
      id="username"
      aria-describedby="error"
      style="display: block;">
```

◆ **Attributes Breakdown**

- `class="input r4 wide mb16 mt8 username" → By.className("input r4 wide mb16 mt8 username")`
- `name="username" → By.name("username")`
- `id="username" → By.id("username")`

◆ **4. Handling Common Elements**

✓ **Input Boxes**

`driver.findElement(By.id("username")).sendKeys("admin@salesforce.com");`

✓ **Password Field**

`driver.findElement(By.id("password")).sendKeys("secret123");`

✓ **Buttons**

`driver.findElement(By.id("login")).click();`

✓ **Dropdowns**

`Select countryDropdown = new Select(driver.findElement(By.id("country")));`
`countryDropdown.selectByVisibleText("India");`

✓ **Radio Buttons**

`driver.findElement(By.id("genderMale")).click();`

✓ **Checkboxes**

`driver.findElement(By.id("subscribe")).click();`

◆ **5. Iframes – Page Inside a Page**

- **Definition:** An `<iframe>` is an HTML document embedded within another HTML document.
- **Switching to iframe:**
- `driver.switchTo().frame("iframeName");`
- **Returning to main page:**
- `driver.switchTo().defaultContent();`

◆ **6. Primitive vs Non-Primitive in Selenium Context**

Type	Description	Example
Primitive	Basic data types (int, boolean)	<code>int count = 5;</code>
Non-Primitive	Objects, classes, interfaces	<code>String name = "Clifford";</code>
Method Return	Can return either type	<code>public int getCount() or public String getName()</code>

◆ **7. End-to-End Login Example**

`WebDriver driver = new ChromeDriver();`
`driver.get("https://login.salesforce.com");`

`driver.findElement(By.id("username")).sendKeys("admin@salesforce.com");`
`driver.findElement(By.id("password")).sendKeys("secret123");`
`driver.findElement(By.id("Login")).click();`

Section 12: Selenium Element Handling: Dropdowns, Iframes, Checkboxes, Radio Buttons, Alerts & Actions

◆ 1. Dropdowns

✓ Standard Dropdowns

Handled using the Select class in Selenium:

🔗 [Select API Documentation](#)

```
WebElement dropdown = driver.findElement(By.id("employeeRange"));
Select select = new Select(dropdown);
select.selectByVisibleText("21 - 200 employees");
```

✓ Multi-Select Operations

```
select.selectByIndex(1);
select.selectByValue("100");
select.deselectByVisibleText("21 - 200 employees");
select.deselectAll();
```

⚠ Dynamic Locators

- Avoid using unstable attributes like data-field-map or auto-generated IDs.
- Prefer id, name, or stable xpath/cssSelector.

◆ 2. Iframes

✓ What is an Iframe?

An <iframe> is an HTML document embedded inside another HTML document.

✓ Switching to Iframe

```
driver.switchTo().frame("frameName"); // or index or WebElement
```

✓ Returning to Main Page

```
driver.switchTo().defaultContent();
```

⚠ Common NoSuchElementException Causes

1. Element not present
2. Incorrect locator
3. Element inside iframe (not switched)

◆ 3. Checkboxes & Radio Buttons

✓ Click Action

```
driver.findElement(By.id("subscribe")).click(); // Checkbox
driver.findElement(By.id("genderMale")).click(); // Radio button
```

🔗 [Checkbox Example](#)

🔗 [Radio Button Example](#)

◆ 4. JavaScript Alerts

✓ Alert Handling

Once a JS alert appears, no other element can be interacted with until it's handled.

```
Alert alert = driver.switchTo().alert();
alert.accept(); // Click OK
alert.dismiss(); // Click Cancel
alert.getText(); // Read alert message
alert.sendKeys("data"); // Enter text (if prompt alert)
```

🔗 [Alert API Documentation](#)

◆ 5. Links & Actions

✓ Link Example

```
<a class="auth-wizard__repair-link" href="javascript:void(0)">Reset</a>
```

✓ Locator Strategy

```
driver.findElement(By.linkText("Reset")).click();  
driver.findElement(By.partialLinkText("Res")).click();
```

✓ Common Actions

Element Type	Action Method
Button	click()
Checkbox	click()
Radio Button	click()
Input Box	sendKeys("value")
Dropdown	Select class
Alert	switchTo().alert()